TrainSight API Documentation

TrainSight is a real-time train tracking system that combines GPS data from multiple sources (crowdsourced, authoritative devices) with a schedule-based physics-aware simulation engine to provide accurate train positions and status information.

Base URL

```
https://core.trainsight.app
```

Authentication

Integration Tokens

Most endpoints require a valid integration token. Include it in the request header:

```
Authorization: Bearer <token>
```

For WebSocket connections, pass the token in the `auth` object on connect.

REST Endpoints

Root

`GET /`

Returns API service information.

Response:

```
{
  "service": "TrainSight API"
}
```

Stations

`GET /api/stations`

Returns list of all stations.

Query Parameters (optional):

- `vehicle_type` (int): Filter by vehicle type (1=train, 0=bus, 2=ferry)

Response:

```
{
  "stations": [
    {
      "code": "LRT-1-001",
      "name": "Baclaran",
      "line": "LRT-1",
      "lat": 14.4329,
      "lng": 120.9512,
      "chainage": 0,
      "vehicle_type": 1,
      "doorside": "Right"
    }
  ]
}
```

Lines

`GET /api/lines`

Returns all transit lines with their geometry data.

Query Parameters (optional):

- `vehicle_type` (int): Filter by vehicle type

Response:

```
{
  "LRT-1": {
    "points": [[lat, lng], ...],
    "cum_dist": [0, 500, 1200, ...]
  }
}
```

Schedule Data

`GET /api/schedule_data`

`GET /api/schedule_data`

Returns schedule data for frontend offline caching.

Response:

```
{
  "operating_hours": [...],
  "fleet_sizes": {"LRT-1": 20, "LRT-2": 10},
  "timetables": {...}
}
```

Fleet Size

`GET /api/fleetSize`

Returns current configured fleet sizes for all lines.

Response:

```
{
  "LRT-1": 20,
  "LRT-2": 10,
  "MRT-3": 10
}
```

GPS Reporting

`POST /api/report_gps`

Submit GPS data from mobile client (crowdsourced).

Headers:

- `Content-Type: application/json`

Request Body:

```
{
  "lat": 14.4500,
  "lng": 120.9700,
  "speed": 30.5,
  "client_id": "device-uuid",
}
```

Response:

```
{"status": "success"}
```

Error Codes:

- 400: Invalid GPS data
- 401: Missing client_id
- 429: Rate limited (train already has 100+ users)

`POST /api/v1/gps/ingest`

Low-latency GPS ingestion endpoint (v1 API).

Request Body:

```
{
  "lat": 14.4500,
  "lng": 120.9700,
  "speed": 30.5,
  "client_id": "client-uuid",
  "vehicle_type": 1,
  "token": "auth-token",
  "is_authoritative": false
}
```

Response:

```
{"status": "success"}
```

Herald (Authoritative) Updates

`POST /api/herald`

Submit GPS data from authorized Herald devices (train-installed).

Request Body:

```
{
  "device_id": "H-001",
  "token": "auth-token",
  "latitude": 14.4500,
  "longitude": 120.9700,
```

```
  "speed": 40.0,
  "direction": 1,
  "line_name": "LRT-1",
  "status": 1
}
```

Response:

```
{"status": "success"}
```

Train Data

`POST /api/data/trains`

Get current train positions.

Request Body:

```
{
  "api_key": "valid-key",
  "user_id": "user-uuid"
}
```

Response:

```
{
  "status": "success",
  "trains": [
    {
      "train_id": "uuid",
      "line": "LRT-1",
      "lat": 14.4500,
      "lng": 120.9700,
      "speed": "40.0 km/h",
      "status": "Live",
      "direction": "Southbound"
    }
  ]
}
```

WebSocket Events

Connection

Connection

```
const socket = io('https://core.trainsight.app', {
  auth: { token: 'integration-token' }
});
```

Events

`ingest_gps_v1`

Low-latency GPS ingestion via WebSocket.

```
socket.emit('ingest_gps_v1', {
  lat: 14.4500,
  lng: 120.9700,
  speed: 30.5,
  device_id: 'device-uuid',
  vehicle_type: 1,
  is_authoritative: false
});
```

Train Object Schema

```
{
  train_id: string,           // Unique identifier
  location: { lat, lng },      // Current position
  line: string,               // Line name (e.g., "LRT-1")
  chainage: number,           // Distance along track in meters
  direction: number,          // 1 or -1
  direction_name: string,     // "Northbound", "Southbound", etc.
  heading: number,            // Compass heading in degrees
  speed: string,              // "40.0 km/h"
  status: string,             // "Live", "Disrupted", "Arrived", etc.
  source: string,             // "calculated", "herald", "schedule"
  is_disrupted: boolean,      // Disruption flag
  is_dwelling: boolean,       // Stopped at station
  next_station: string,       // Next station code
  next_station_name: string,  // Next station name
  eta_next_station: string,   // "3 mins to Roosevelt"
  door_side: string,          // "Left", "Right", "Both"
  occupancy_percent: number,   // 0-100
  loading_status: string,     // "Light", "Moderate", "Heavy"
  gps_user_count: number,     // Number of GPS users on this train
  last_update: string,        // "02:30 PM"
  timestamp: number           // Unix timestamp
}
```

Data Sources

Source	Description	Priority
`herald`	Train-installed GPS devices	Highest
`calculated`	Crowdsourced GPS aggregation	Medium
`schedule`	Simulated schedule-based trains	Lowest

When real GPS data exists for a line, scheduled trains are suppressed to avoid duplication.

Rate Limits

- GPS reports: Rate limited when a train reaches 100 users
- Crowdsourced updates timeout after 20 seconds of inactivity
- Scheduled trains despawn after 5 minutes without updates

Error Responses

```
{
  "status": "error",
  "message": "Error description"
}
```

Common status codes:

- 200: Success
- 400: Bad Request
- 401: Unauthorized
- 403: Forbidden
- 429: Rate Limited
- 500: Server Error